Navigator

Chikishev Mikhail, SE-102
Mikhail Sharmanov, SE-103
Andreev Alexey, SE-103
Egor Yemelyanov, SE-103

March 31, 2026

Contents

1 Introduction **2**

1.1 Project Rationale 2

2 Theoretical Framework **3**

2.1 Analysis of Current Challenges 3

2.2 Core Development Principles 3

2.3 Panoramic Content Visualization Methods 4

2.4 Routing Fundamentals 4

2.5 Distributed Communication Protocols 4

2.6 Information Security 5

3 Practical Implementation **7**

3.1 General Architecture Overview 7

3.1.1 Infrastructure Characteristics and Design Principles . . 8

3.1.2 Technology Stack Components 8

3.2 Backend Implementation 9

3.2.1 Component Initialization and Deployment Process . . . 10

3.2.2 Transport Layer and Request Processing 10

3.2.3 Abstraction Layers: Services and Repositories 11

3.2.4 Background Tasks and Asynchronous Processing 11

3.3 DevOps Methodology and Information Security 12

3.3.1 Continuous Integration (CI) Pipeline and Quality Assurance 12

3.3.2 Containerization and Runtime Security 12

3.3.3 Monitoring and Network Perimeter Protection 13

1. Introduction

1.1 Project Rationale

The project aims to develop a replacement for the existing navigation service map.csu.ru. The primary driver for development is the need to implement modern 360-degree panoramic imaging technologies to enhance the visualization of the university environment. During the conceptual discussion, it was decided to discontinue support for the current codebase in favor of developing a fundamentally new application. This decision is based on the following factors:

1. low cost-effectiveness of maintaining the previous version due to accumulated technical debt and architectural complexity;
2. the necessity of implementing a modern technology stack to ensure scalability and performance.

The project objective is to develop a multifunctional ecosystem that addresses key university digitalization challenges and improves the Quality of Life (QoL) for students and staff. The implementation roadmap includes:

- development of a cross-platform interactive map editor with 3D imagery support, event and schedule integration, and shortest-path routing algorithms;
- integration of an ejabberd server and a web client to facilitate internal communication;
- development of data import tools (parsers) or manual schedule entry interfaces.

2. Theoretical Framework

2.1 Analysis of Current Challenges

Currently, the academic process is associated with several organizational complexities. These include the absence of a unified secure environment for real-time communication between students and faculty, the decentralization of university information resources, and the low efficiency of notification channels.

The software product under development is designed to aggregate the necessary services into a single interface, thereby increasing the transparency and accessibility of the educational environment.

2.2 Core Development Principles

The key priorities in system design are security, performance, portability, and maintainability. These requirements dictated the choice of the technology stack. Several languages, including Python, JavaScript, and C#, were excluded from consideration as they did not meet the performance criteria or the specific architectural requirements of the project.

Furthermore, a significant objective for the development team is to gain hands-on experience with high-performance systems and to master industry-relevant skills.

2.3 Panoramic Content Visualization Methods

To achieve a full sense of presence and indoor navigation, 360-degree panoramic imaging technologies are employed. The key task in the software processing of such content is the transformation of spherical data into a flat image and its subsequent re-projection within the user interface.

The primary standard here is the equirectangular projection, where the coordinates of each point on the sphere (θ, ϕ) are mapped to the Cartesian coordinates of the image pixels (x, y).

Within the scope of this project, 360-degree map visualization is implemented using the specialized rendering library `three.js`, which maps the equirectangular texture onto the inner surface of a virtual sphere or cube. To optimize client-side performance, a segmented loading (tiling) technique is applied, avoiding excessive RAM consumption when handling ultra-high-resolution images (8K and above).

2.4 Routing Fundamentals

The system is based on a mathematical model of a weighted directed graph (where each edge has a value), constructed according to the principle of Poincaré duality. In this model, 3D indoor volumes (rooms, corridors) are transformed into nodes (0D), while the connecting surfaces (doors) become edges (1D). The integration of the IndoorGML standard (describing 3D topology and XYZ coordinates) allows for dynamic network filtering via the `pgRouting` extension. This enables the calculation of the shortest path using Dijkstra's algorithm on an up-to-date subgraph that accounts for the physical connectivity of floors in real-time.

2.5 Distributed Communication Protocols

To establish a unified interaction environment for educational process participants, the XMPP (Extensible Messaging and Presence Protocol) was se-

lected. Unlike proprietary solutions, XMPP is based on open XML standards and provides high extensibility through XEP (XMPP Extension Protocols) specifications.

The choice of the `ejabberd` server is driven by its fault tolerance, robust API, and ability to handle a large number of concurrent connections, which is critical for university-scale operations. In the system architecture, `ejabberd` serves as a module providing:

- real-time instant messaging;
- isolation of personal social networks from the learning process;
- multi-user conference management for organizing study groups;
- user notification services.

The protocol integration ensures a communication channel decoupled from personal data and eliminates dependency on a specific client, while maintaining traffic and message encryption and excluding external infrastructure.

2.6 Information Security

Data protection is addressed at all stages of the Software Development Life Cycle (SDLC). The choice of the Go programming language is justified by its memory safety mechanisms. To minimize attack vectors, the following solutions are implemented:

- use of ORM to prevent SQL injections;
- application of Anubis for protection against automated attacks (bots);
- implementation of specialized Middleware;
- use of JWT for user authorization;
- deployment in "distroless" containers to reduce the attack surface;
- integration of static analyzers and Quality Gates into the CI/CD process.

This approach implements the "Shift Left Security" concept, allowing for the identification and remediation of vulnerabilities at the early stages of development.

3. Practical Implementation

3.1 General Architecture Overview

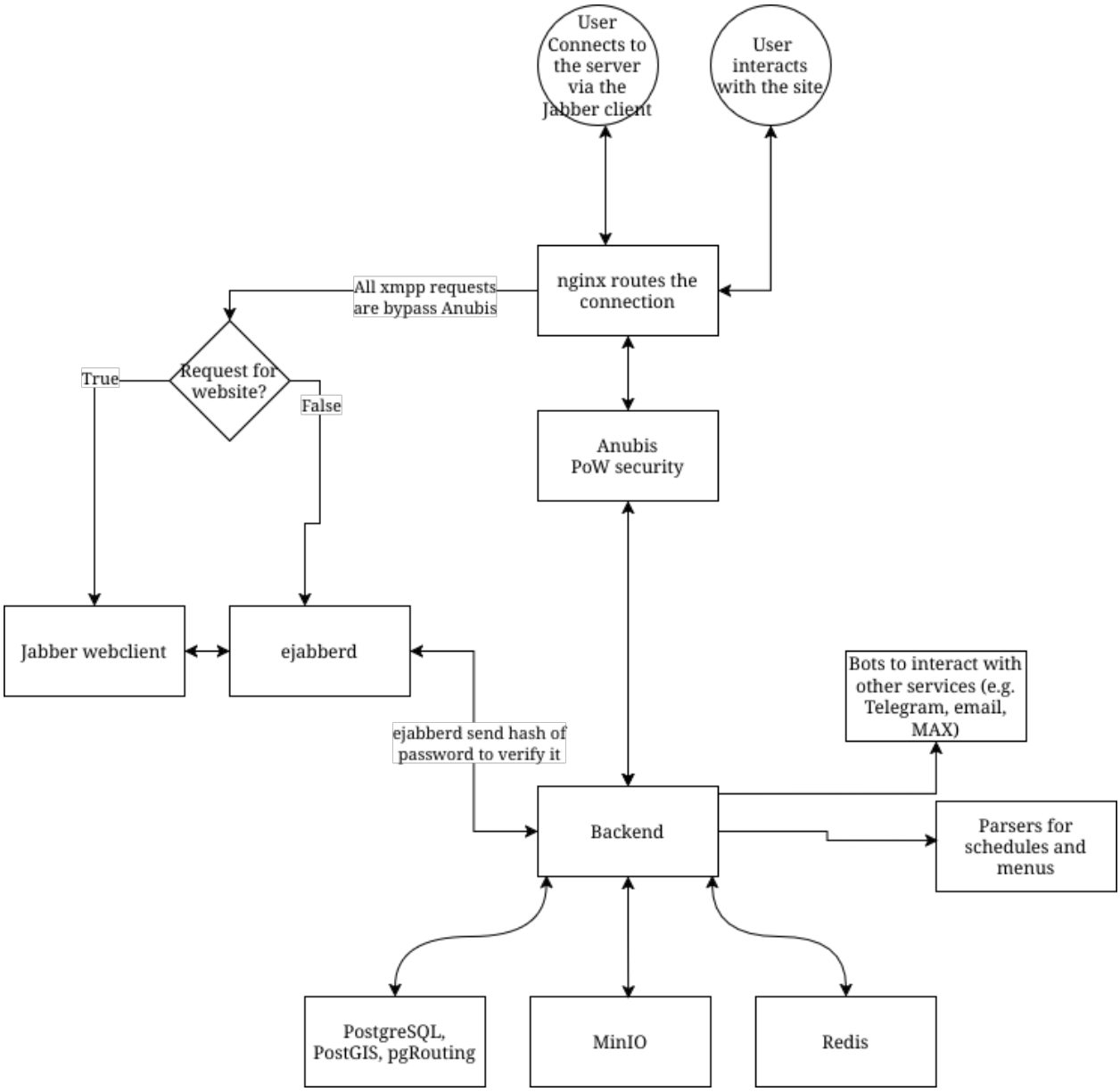

Figure 3.1: General architecture diagram

Let us consider the general architecture presented in Fig. 3.1. The designed system architecture is dictated by the specifics of the target infrastructure — deployment within an isolated virtual machine (VM) with limited computing resources (1 vCPU, 1 GB RAM). These conditions impose several constraints on the use of orchestration tools and necessitate a transition to a hybrid software development model.

3.1.1 Infrastructure Characteristics and Design Principles

The choice of a `docker-compose` based deployment environment limits horizontal scaling capabilities in the short term; however, it ensures high autonomy in system resource management. The core architectural principles include:

- **Service Consolidation:** Minimizing the number of containers to reduce virtualization overhead under resource-constrained conditions.
- **Hybrid Monolith:** The core business logic is implemented within a single service (Backend), which simplifies inter-node communication while maintaining modularity for connecting external components (parsers and bots).
- **Stateless Approach:** The backend is designed without session state persistence, ensuring the system's potential readiness for migration to a clustered environment (e.g., Kubernetes) for future scaling.

3.1.2 Technology Stack Components

1. Networking and Security (NGINX, Anubis)
To handle traffic routing and ensure information security, a combination of the NGINX web server and the Anubis protection system will be utilized. This solution is critical for preventing system performance degradation caused by unauthorized traffic spikes (automated attacks) under limited VM resource conditions.

2. Storage and Geospatial Analysis Layer (PostgreSQL, PostGIS,

pgRouting)
PostgreSQL has been selected as the primary DBMS. Its **PostGIS** extension provides efficient storage and processing of spatial data. To implement navigation logic, the **pgRouting** extension will be integrated, offering tools to execute shortest-path algorithms directly on the database side. This allows for offloading the main service by delegating complex graph computations to the optimized DBMS engine.

3. Communication Subsystem (Ejabberd and XMPP Client)
The `ejabberd` server will be used to facilitate user interaction. A key feature of this integration is the delegation of user authentication to the main backend. The use of the OAuth protocol enables a Single Sign-On (SSO) concept, eliminating the need to duplicate and synchronize credentials across multiple data stores. The selection of a specific XMPP web client is currently under investigation.

4. Object Storage and Caching (MinIO, Redis)
Static content (map tiles, 3D panoramas) will be stored in the S3-compatible `MinIO` object storage. `Redis` will be employed for task queue management and real-time data caching, significantly improving system response times.

5. External Integration Services (Parsers and Bots)
Data collection modules (schedule parsers) are implemented in Python to ensure flexibility in processing structured information. For chat-bot development, priority is given to the Go language to minimize resource consumption and ensure native compatibility with the core backend architecture; however, Python may be used when specific protocol support is required.

3.2 Backend Implementation

The backend of the application will be developed in Go using a multi-layered architecture, which ensures loose coupling of components and simplifies system testing. This section examines the application lifecycle, dependency management mechanisms, and key subsystems based on Fig. 3.2.

3.2.1 Component Initialization and Deployment Process

The application startup is a strictly deterministic sequence of resource and service initialization stages. The deployment algorithm includes the following steps:

1. **Configuration:** Using the `Viper` library to load environment parameters from environment variables and configuration files. This allows for flexible adjustment of settings (ports, database addresses, access keys) without rebuilding the binary file.

2. **Database State Management:** Establishing a connection to PostgreSQL and executing automatic migrations using `golang-migrate`. This guarantees data schema consistency, including the `PostGIS` and `pgRouting` extensions.

3. **Caching and Queue Initialization:** Creating clients for `Redis` and `Asynq`, which are required for handling background tasks and transient data.

4. **Dependency Injection (DI):** Implementing the IoC (Inversion of Control) pattern. All objects (repositories, services, handlers) receive their dependencies explicitly through constructors. This approach eliminates hidden dependencies within components and facilitates unit testing.

5. **Transport Layer Startup:** Initializing HTTP server routes, the WebSocket hub, and activating background workers.

3.2.2 Transport Layer and Request Processing

The `Gin` framework was selected as the primary HTTP framework, characterized by high performance and a robust *middleware* ecosystem. The lifecycle of an incoming request includes:

- Passing through a chain of *middleware* (authentication, logging, attack protection).

- Routing to the corresponding handler.
- Invoking business logic through the service layer and returning a structured response.

To ensure interactivity and real-time event processing (e.g., during map editing or receiving notifications), a **WebSocket Hub** subsystem has been implemented. It manages active connections and broadcasts updates between clients, minimizing latency compared to conventional HTTP requests.

3.2.3 Abstraction Layers: Services and Repositories

The application architecture is divided into logical levels to isolate business rules from infrastructure implementation details:

1. **Service Layer:** Encapsulates the core business logic of the project. Services operate through repository interfaces and external clients (`Redis`, `MinIO`), remaining independent of the specific storage implementation. This layer also handles interaction with **pgRouting** to calculate optimal paths based on the provided coordinates.
2. **Data Access Layer (DAL):** Implemented via the "Repository" pattern. Repositories hide the details of SQL queries and the specifics of the `GORM` ORM.
3. **Interface Abstraction:** The use of interfaces allows for easy replacement of real database connections with *mock* objects during testing, which is critical for ensuring code stability.

3.2.4 Background Tasks and Asynchronous Processing

The `Asynq` library is used to perform resource-intensive operations, such as schedule parsing or bulk geospatial data processing. Tasks are placed into a `Redis` queue and processed by dedicated workers. This allows the main HTTP server to remain responsive by not blocking the execution flow during heavy computations.

3.3 DevOps Methodology and Information Security

A modern Continuous Integration and Continuous Delivery (CI/CD) cycle centered on the "Shift Left Security" concept has been implemented within the project. The primary objective of this approach is to identify defects and vulnerabilities at the earliest possible stages of development.

3.3.1 Continuous Integration (CI) Pipeline and Quality Assurance

The code build and verification process will be automated using the **Jenkins** automation server. The lifecycle of changes in the codebase is strictly regulated and includes the following stages:

1. **Automated Testing:** Every *PR* (Pull Request) triggers a suite of unit and integration tests. Passing all tests is a mandatory prerequisite for further code analysis.

2. **Static Analysis and Quality Gates:** The source code undergoes deep scanning in the **SonarQube** system. The check includes searching for potential vulnerabilities, assessing technical debt, and analyzing code coverage. Branch merging is permitted only upon successfully passing the established *Quality Gates*.

3. **Mandatory Code Review:** The branching policy prohibits direct pushes to the `main` branch. Any change requires review and formal approval from at least one other member of the development team.

3.3.2 Containerization and Runtime Security

To minimize attack vectors during the software product's operational phase, containerization technology utilizing specialized images will be employed.

- **Distroless Containers:** Unlike standard base images (e.g., those based on Debian or Alpine), *distroless* images contain exclusively the

application and its runtime dependencies. They lack package managers, shells, and standard OS utilities. This significantly reduces the "attack surface" and eliminates the possibility of an attacker executing arbitrary scripts within the container.

- **Multi-stage Builds:** The Go application is compiled in an intermediate container, after which the resulting binary is copied into a minimalist final image.

3.3.3 Monitoring and Network Perimeter Protection

The infrastructure layer is supplemented with proactive defense tools:

- **Middleware Filtering:** Implementation of an intermediary request-processing layer to validate incoming data and prevent common attacks (SQL injections, XSS).
- **Anubis:** Utilizing the Anubis system in conjunction with NGINX allows for effective blocking of automated malicious traffic and botnets, maintaining VM resource availability for legitimate users.

This suite of measures ensures that only verified, tested, and secured code that meets established security standards reaches the runtime environment.

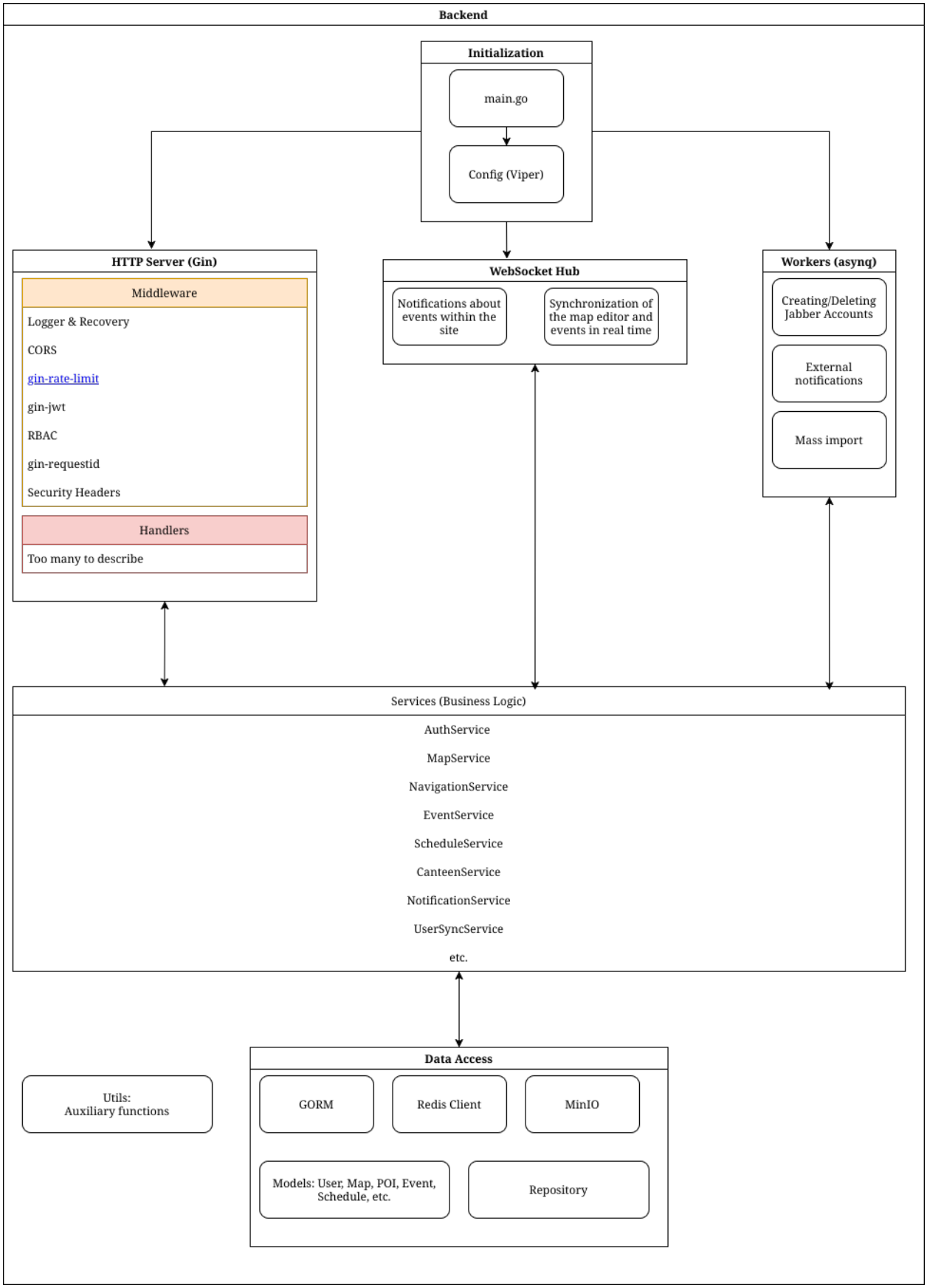

Figure 3.2: Backend block diagram

Conclusion

In the course of this work, the architecture of a multifunctional university navigation ecosystem was designed. The developed solution will not only replace outdated visualization tools but also establish a foundation for a unified digital space within the university.

The key achievements of the project include:

- **Technological Modernization:** The transition to the Go language and a microservice-oriented approach ensured high system performance even under constrained server resources.
- **Navigation Innovation:** The implementation of 360-degree panoramas in conjunction with the `pgRouting` mathematical framework and graph-based models allowed for the creation of an intuitive and accurate indoor routing tool.
- **Security and Reliability:** Applying the "Shift Left Security" concept, utilizing *distroless* containers, and maintaining a rigorous CI/CD pipeline guarantee the protection of user data and system resilience against external threats.
- **Scalability:** The selected technology stack and the *stateless* backend architecture allow for seamless project scaling as load increases or functionality expands.

Once the software product is developed, it will comply with modern development standards. Upon deployment within the university infrastructure, it will provide a significant improvement in the quality of interaction between students, faculty, and the university's digital environment.